Automated Lower Bounds for Small Matrix Multiplication Complexity over Finite Fields

Chengu Wang
wangchengu@gmail.com

Abstract

We develop an automated framework for proving lower bounds on the bilinear complexity of matrix multiplication over finite fields. Our approach systematically combines symmetry reduction, restriction space enumeration, and recursive substitution arguments, culminating in efficiently verifiable proof certificates. Using this framework, we obtain several new lower bounds for various small matrix formats. Most notably, we prove that the bilinear complexity of multiplying two 3×3 matrices over $\mathbb{F}_2$ is at least 20, improving upon the longstanding lower bound of 19 (Bläser 2003). Our computer search discovers this proof in approximately 1.5 hours on a laptop, and the resulting certificate can be verified in seconds.

1 Introduction

Matrix multiplication is one of the central problems in algebraic complexity theory. Since Strassen showed that two 2×2 matrices can be multiplied using only seven scalar multiplications instead of eight [13], the study of upper and lower bounds has advanced rapidly.

In the bilinear and non-commutative setting, one asks for the minimum number of scalar products needed to express the entries of the product of two matrices as bilinear forms in the input entries. Equivalently, one may ask for the tensor rank of the corresponding matrix multiplication tensor. In a ground field $\mathbb{K}$, the matrix multiplication tensor $\langle l, m, n\rangle$ can be viewed naturally as an element of the tensor product space $\mathbb{K}^{l\times m} \otimes \mathbb{K}^{m\times n} \otimes \mathbb{K}^{n\times l}$ (or its duals). These viewpoints coincide, and we denote the tensor rank of this common quantity by $\mathbf{R}(\langle l, m, n\rangle)$.

Small matrix formats play a particularly important role. On the algorithmic side, efficient bilinear algorithms for small formats can be composed recursively to obtain algorithms for larger matrices. On the lower-bound side, small formats are among the few settings where one can still hope to obtain exact ranks or nearly sharp bounds, making them a natural laboratory for understanding the strengths and limitations of current methods. Among these, the tensor $\langle 3, 3, 3\rangle$ is especially prominent: it is the smallest square format for which the best-known upper and lower bounds have remained separated.

1.1 Previous Lower Bounds

The smallest nontrivial case is $\langle 2,2,2\rangle$. Strassen [13] gave the upper bound $\mathbf{R}(\langle 2,2,2\rangle) \le 7$ over $\mathbb{Z}$, and Winograd [14] proved the matching lower bound $\mathbf{R}(\langle 2,2,2\rangle) \ge 7$ over $\mathbb{Q}$. Thus the exact value is known in this case.

Beyond 2×2 multiplication, Hopcroft and Kerr [7] proved lower bounds for several small formats over $\mathbb{F}_2$, including

$$\mathbf{R}(\langle 2,2,n\rangle) \ge \left\lceil \frac{7n}{2} \right\rceil \qquad \text{and} \qquad \mathbf{R}(\langle 2,3,3\rangle) \ge 15,$$

with the latter matching their upper bound. They also mentioned the bound $\mathbf{R}(\langle 2,3,4\rangle) \ge 19$ over $\mathbb{F}_2$, but without a complete proof.

By meticulously analyzing the linear independence of the bilinear forms generated by the matrix product, Bshouty [4] proved that

$$\mathbf{R}(\langle n,n,n\rangle) \ge \frac{5}{2}n^2 - o(n^2)$$

over $\mathbb{F}_2$.

A decade later, Bläser dramatically established three influential families of lower bounds in general fields. First, he proved

$$\mathbf{R}(\langle l,m,n\rangle) \ge lm + mn + l - m + n - 3 \qquad (n \ge l \ge 2)$$

over algebraically closed fields [2]. It implies the same lower bound over $\mathbb{Q}$ and $\mathbb{F}_2$. Second, he proved

$$\mathbf{R}(\langle n,n,n\rangle) \ge \frac{5}{2}n^2 - 3n$$

over arbitrary fields [1]. Third, he later obtained the stronger bound

$$\mathbf{R}(\langle n,m,n\rangle) \ge 2mn + 2n - m - 2 \qquad (m \ge n \ge 3)$$

over arbitrary fields [3], which yields in particular

$$\mathbf{R}(\langle 3,3,3\rangle) \ge 19.$$

On the upper-bound side, Laderman [9] constructed a rank-23 decomposition for $\langle 3,3,3\rangle$ over $\mathbb{Z}$. Thus, the exact rank of $\langle 3,3,3\rangle$ has remained unknown between 19 and 23. This makes $\langle 3,3,3\rangle$ a natural target for research.

1.2 Our Lower Bounds

We prove that the tensor rank of the matrix multiplication tensor for two 3×3 matrices is at least 20 over $\mathbb{F}_2$; equivalently, any bilinear algorithm for multiplying two 3×3 matrices over $\mathbb{F}_2$ must use at least 20 scalar multiplications. This improves upon the longstanding lower bound of 19 established by Bläser in 2003 [3].

Theorem 1. $\mathbf{R}(\langle 3,3,3\rangle) \geq 20$ *over* $\mathbb{F}_2$.

To achieve this, we developed an automated framework designed to prove lower bounds for small matrix multiplication tensors over finite fields. This framework relies on four core computational ingredients: the systematic enumeration of linear restriction spaces on a single matrix factor, the aggressive pruning of the search space modulo the natural symmetries of matrix multiplication, the recursive application of substitution-style lower-bound arguments via a backtracking search, and the generation of independently verifiable proof certificates. The efficiency of this approach is demonstrated by our $\langle 3,3,3\rangle$ result: the search algorithm constructs the proof in just 1.5 hours on a laptop, while the generated certificate can be deterministically verified on the same laptop in seconds.

Beyond the 3×3 case, our method yields several further new lower bounds over $\mathbb{F}_2$, demonstrating its general applicability to various matrix dimensions.

Theorem 2. $\mathbf{R}(\langle 2,3,4\rangle) \geq 19$ *over* $\mathbb{F}_2$.

Theorem 3. $\mathbf{R}(\langle 3,3,4\rangle) \geq 25$ *over* $\mathbb{F}_2$.

Theorem 4. $\mathbf{R}(\langle 3,4,4\rangle) \geq 29$ *over* $\mathbb{F}_2$.

Our lower bound for $\langle 2,3,4\rangle$ completes the proof that was briefly mentioned but skipped by Hopcroft and Kerr in 1971 [7]. Our bounds for $\langle 3,3,4\rangle$ and $\langle 3,4,4\rangle$ further confirm that the method scales to many matrix sizes.

The table below summarizes the tensor rank lower and upper bounds over $\mathbb{F}_2$, highlighting our contributions in boxes.

Matrix Size	Previous LB	Our LB	UB
$\langle 2,2,2\rangle$	7 [14, 7]	7	7 [13]
$\langle 2,2,3\rangle$	11 [7]	11	11 ($\langle 2\times 2\times 2 : 7\rangle + \langle 2\times 2\times 1 : 4\rangle$)
$\langle 2,2,4\rangle$	14 [7]	14	14 ($\langle 2\times 2\times 2 : 7\rangle + \langle 2\times 2\times 2 : 7\rangle$)
$\langle 2,3,3\rangle$	15 [7]	15	15 [7]
$\langle 2,3,4\rangle$	18 [2], 19? [7]	$\boxed{19}$	20 [7]
$\langle 3,3,3\rangle$	19 [3]	$\boxed{20}$	23 [9]
$\langle 3,3,4\rangle$	24 [3]	$\boxed{25}$	29 [11]
$\langle 3,4,4\rangle$	28 [2]	$\boxed{29}$	38 [11]

While this paper focuses on $\mathbb{F}_2$, the underlying techniques can be adapted to other finite fields.

The rest of the paper is organized as follows. Section 2 introduces the symmetries of matrix multiplication tensors. Section 3 describes the enumeration of restriction orbits. Section 4 presents a toy example proving $\mathbf{R}(\langle 2,2,2\rangle) \geq 7$ over $\mathbb{F}_2$ to illustrate the method. Section 5 formalizes the lower-bound techniques utilized in the automated search. Section 6 explains the structure of the proof certificates and their verification. Section 7 illustrates applications of our method to concrete tensor formats. Finally, Section 8 concludes the paper and discusses future work.

2 Symmetries

The symmetries of the matrix multiplication tensor have been well studied [6, 5].

Consider the matrix multiplication problem $AB = C^T$, where $A \in \mathbb{K}^{l\times m}$, $B \in \mathbb{K}^{m\times n}$, and $C \in \mathbb{K}^{n\times l}$. Let $a_{i,j}$, $b_{j,k}$, and $c_{k,i}$ denote the formal basis variables for these respective matrix spaces. The matrix multiplication tensor $\langle l, m, n\rangle$ is formally defined as:

$$\langle l, m, n\rangle = \sum_{i=0}^{l-1}\sum_{j=0}^{m-1}\sum_{k=0}^{n-1} a_{i,j} \otimes b_{j,k} \otimes c_{k,i} \in \mathbb{K}^{l\times m} \otimes \mathbb{K}^{m\times n} \otimes \mathbb{K}^{n\times l}.$$

For any invertible matrices $P \in \mathrm{GL}_l(\mathbb{K})$, $Q \in \mathrm{GL}_m(\mathbb{K})$, and $R \in \mathrm{GL}_n(\mathbb{K})$, the transformation $(PAQ^{-1})(QBR^{-1}) = (RCP^{-1})^T$ demonstrates an equivalence under base change. This group action preserves the tensor rank, a property we refer to as the *sandwich symmetry*.

The next symmetry is the *cyclic symmetry*. The tensor rank is invariant under the cyclic permutation of the three tensor factors: $\mathbf{R}(\langle l, m, n\rangle) = \mathbf{R}(\langle m, n, l\rangle)$.

For square matrices where $l = m = n$, applying the transpose operation yields $B^T A^T = C^T$. Combined with the cyclic symmetry, this gives us the *transpose symmetry*:

$$\mathbf{R}\left(\sum_{i=0}^{n-1}\sum_{j=0}^{n-1}\sum_{k=0}^{n-1} a_{i,j} \otimes b_{j,k} \otimes c_{k,i}\right) = \mathbf{R}\left(\sum_{i=0}^{n-1}\sum_{j=0}^{n-1}\sum_{k=0}^{n-1} a_{j,i} \otimes c_{i,k} \otimes b_{k,j}\right).$$

3 Restrictions

Several recent studies have investigated structured matrix multiplication. Khoruzhii et al. [8] leveraged a flip graph search to find upper bounds for triangular, symmetric, and skew-symmetric matrices. Solomonik et al. [12] focused on contracting symmetric tensors like symmetric matrix-vector products, and Newhouse et al. [10] developed an optimized symmetric matrix multiplication kernel for NVIDIA GPUs.

We consider the matrix multiplication problem subject to linear restrictions on the factor space corresponding to the matrix A. Applying linear restrictions to $a_{i,j}$ confines the matrix multiplication tensor to a subspace. For example, the restrictions $a_{i,j} = a_{j,i}$ constrain A to symmetric matrices, and the restrictions $a_{0,j} = 0$ reduce the problem to the $\langle l-1, m, n\rangle$ format.

Since each restriction on A acts as a linear functional, it can be naturally represented by an $l \times m$ coefficient matrix in the dual space over $\mathbb{F}_2$. Under the standard Frobenius inner product, a matrix A satisfies the restriction if and only if it is orthogonal to this coefficient matrix. For example, for 2×2 matrix A, the restriction $a_{0,1} = 0$ is represented by the matrix $\begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$, and the restriction $a_{0,0} + a_{0,1} = 0$ is represented by the matrix $\begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix}$.

Some sets of restrictions are equivalent because they either define the same linear subspace or are equivalent under the symmetries defined above (sandwich, cyclic, transpose). For example, the restriction sets $\{a_{0,0} = 0,\ a_{0,1} = 0\}$ and $\{a_{0,0} = 0,\ a_{0,0}+a_{0,1} = 0\}$ are equivalent because they define the exact same subspace. The restrictions $\{a_{0,j} = 0 : 0 \leq j < m\}$ and $\{a_{1,j} = 0 : 0 \leq j < m\}$ are equivalent under the sandwich symmetry, as we can simply permute the rows of A by a non-singular matrix P.

Our goal is to enumerate all distinct orbits of equivalent restrictions on A over the finite field $\mathbb{F}_2$. For each orbit, we select a canonical representative. We can determine whether two sets of restrictions define the same subspace by mapping them to their reduced row echelon form (RREF) via Gauss-Jordan elimination. The RREF naturally serves as the lexicographically minimal form for a subspace (depending on the chosen lexicographical ordering of the variables, possibly after reversing row or column orders).

Algorithm 1 Enumerate restriction orbits on $l \times m$ matrix A over $\mathbb{F}_2$ – Slow

Ensure: Returns *orbits*, a lexicographically ordered list containing one minimal representative for each orbit. Each restriction is an $l \times m$ matrix over $\mathbb{F}_2$, or a binary integer of lm bits.

```
orbits ← empty list
SEARCH(empty list)
return orbits

function SEARCH(restrictions)
    if |restrictions| = lm then
        return
    end if
    last ← last element of restrictions (or 0 if empty)
    for restriction = last + 1 to 2^(lm) − 1 do
        if restriction is in subspace of restrictions then
            continue
        end if
        append restriction to restrictions
        if ISCANONICAL(restrictions) then
            append restrictions to orbits
            SEARCH(restrictions)
        end if
        remove last element from restrictions
    end for
end function

function ISCANONICAL(restrictions)
    for each full-rank matrix P and Q^(−1) in random order do
        for transpose in {false, true} if l = m = n do
            new_restrictions ← RREF of {PMQ^(−1) : M ∈ restrictions} (transpose M if transpose)
            if new_restrictions <_lex restrictions then
                return false
            end if
        end for
    end for
    return true
end function
```

Algorithm 1 finds 10 distinct orbits in the $\langle 2, 2, 2\rangle$ case and 496 orbits in the $\langle 3, 3, 3\rangle$ case within a minute. However, we need a faster algorithm for $\langle 3, 4, 4\rangle$.

Algorithm 2 Enumerate restriction orbits on $l \times m$ matrix A over $\mathbb{F}_2$ – Fast

Ensure: Returns *orbits*, a lexicographically ordered list containing one minimal representative for each orbit.

```
visited ← ∅
orbits ← [∅]
SEARCH(empty list)
return orbits

function SEARCH(restrictions)
    if |restrictions| = lm then
        return
    end if
    pivot_set ← { most significant bit position of restriction : restriction ∈ restrictions }
    for restriction ∈ {0,1}^lm with all bits in pivot_set equal to 0 do
        append restriction to restrictions
        if VISIT(restrictions) then
            SEARCH(restrictions)
        end if
        remove last element from restrictions
    end for
end function

function VISIT(restrictions)
    for transpose in {false, true} if l = m = n do
        for each P ∈ GL_l(F_2) do
            left_normalized ← RREF of {PM : M ∈ restrictions} (transpose M if transpose)
            if left_normalized ∈ visited then
                return false
            end if
        end for
    end for
    for each Q ∈ GL_m(F_2) do
        right_normalized ← RREF of {MQ^-1 : M ∈ restrictions}
        Add right_normalized to visited
    end for
    append restrictions to orbits
    return true
end function
```

Algorithm 2 introduces two key optimizations. First, it avoids scanning all 2^{lm} candidates at each node. Instead, it only considers candidate restrictions that have zeros at the pivot positions of the already selected restrictions. Any candidate with a one at an existing pivot position would be functionally redundant, as it would be reduced to zero by Gauss-Jordan elimination against the current basis.

Second, Algorithm 2 trades space for time when verifying if a restriction set is canonical. While Algorithm 1 enumerates all full-rank matrices P and Q, Algorithm 2 uses a set to store the right-normalized forms (the RREF of MQ^{-1}) for each full-rank matrix Q. Consequently, it only needs to enumerate the left-multiplication matrices P to check for canonicity. Roughly speaking, this trades square-root space for square-root time. [1]

Using these optimizations, Algorithm 2 processes the 3×4 matrix A case in 5 minutes, consuming 168 GB of memory to find all 158,426 distinct orbits.

4 A Toy Example: $\mathbf{R}(\langle 2,2,2\rangle) \geq 7$ over $\mathbb{F}_2$

We take $\mathbf{R}(\langle 2,2,2\rangle)$ as a running example to illustrate how our tensor rank lower bound techniques operate in practice.

There are 10 orbits of restrictions on the matrix A over $\mathbb{F}_2$. We analyze them sequentially, from the most restricted to the general case.

Orbit 0: $\{a_{0,0} = 0,\ a_{0,1} = 0,\ a_{1,0} = 0,\ a_{1,1} = 0\}$

$A = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$. The matrix multiplication tensor is $T_0 = 0$, so trivially $\mathbf{R}(T_0) = 0$.

Orbit 1: $\{a_{0,0} = 0,\ a_{0,1} = 0,\ a_{1,0} = 0\}$

$A = \begin{pmatrix} 0 & 0 \\ 0 & a_{1,1} \end{pmatrix}$. The matrix multiplication tensor is $T_1 = a_{1,1} \otimes b_{1,0} \otimes c_{0,1} + a_{1,1} \otimes b_{1,1} \otimes c_{1,1}$.

Lemma 1 (Flattening)**.** *The rank of a trilinear tensor is bounded below by the rank of the matrix obtained by flattening any two of its dimensions into one.*

By flattening the spaces for a and b into a single space x with formal variables $x_{i,j,j',k'} = a_{i,j} \otimes b_{j',k'}$, we reshape the tensor into $x_{1,1,1,0} \otimes c_{0,1} + x_{1,1,1,1} \otimes c_{1,1}$. Viewed as a matrix mapping, it has a matrix rank of 2. By Lemma 1, $\mathbf{R}(T_1) \geq 2$.

Orbit 2: $\{a_{0,0} = 0,\ a_{0,1} + a_{1,0} = 0,\ a_{1,1} = 0\}$

$A = \begin{pmatrix} 0 & a_{0,1} \\ a_{0,1} & 0 \end{pmatrix}$. The tensor is $T_2 = a_{0,1} \otimes b_{0,0} \otimes c_{0,1} + a_{0,1} \otimes b_{0,1} \otimes c_{1,1} + a_{0,1} \otimes b_{1,0} \otimes c_{0,0} + a_{0,1} \otimes b_{1,1} \otimes c_{1,0}$. Flattening a and b yields $x_{0,1,0,0} \otimes c_{0,1} + x_{0,1,0,1} \otimes c_{1,1} + x_{0,1,1,0} \otimes c_{0,0} + x_{0,1,1,1} \otimes c_{1,0}$.

[1] Actually, we can have other time-space trade-offs by putting P, Q and transpose in either the time or space side.

This has a matrix rank of 4, yielding $\mathbf{R}(T_2) \geq 4$.

Orbit 3: $\{a_{0,0} = 0,\ a_{0,1} = 0\}$

$A = \begin{pmatrix} 0 & 0 \\ a_{1,0} & a_{1,1} \end{pmatrix}$. The tensor is $T_3 = a_{1,0} \otimes b_{0,0} \otimes c_{0,1} + a_{1,0} \otimes b_{0,1} \otimes c_{1,1} + a_{1,1} \otimes b_{1,0} \otimes c_{0,1} + a_{1,1} \otimes b_{1,1} \otimes c_{1,1}$. Flattening a and b yields $x_{1,0,0,0} \otimes c_{0,1} + x_{1,0,0,1} \otimes c_{1,1} + x_{1,1,1,0} \otimes c_{0,1} + x_{1,1,1,1} \otimes c_{1,1}$. This has a matrix rank of 4, yielding $\mathbf{R}(T_3) \geq 4$.

Orbit 4: $\{a_{0,0} = 0,\ a_{0,1} + a_{1,0} = 0\}$

$A = \begin{pmatrix} 0 & a_{0,1} \\ a_{0,1} & a_{1,1} \end{pmatrix}$. The tensor is $T_4 = a_{0,1} \otimes b_{0,0} \otimes c_{0,1} + a_{0,1} \otimes b_{0,1} \otimes c_{1,1} + a_{0,1} \otimes b_{1,0} \otimes c_{0,0} + a_{0,1} \otimes b_{1,1} \otimes c_{1,0} + a_{1,1} \otimes b_{1,0} \otimes c_{0,1} + a_{1,1} \otimes b_{1,1} \otimes c_{1,1}$.

For this orbit, the Flattening Lemma only provides a lower bound of 4. To push the bound higher, we rely on a technique introduced by Hopcroft and Kerr [7].

Lemma 2 (Lemma 2 in [7])**.** *Let $F = \{f_0, f_1, \ldots, f_{s-1}, \ldots, f_{t-1}\}$ be a set of expressions, where $f_0, f_1, \ldots, f_{s-1}$ are linearly independent, and each can be expressed as a single product. If F can be computed with p multiplications, then there exists a bilinear algorithm for F with p multiplications in which s of the multiplications are exactly $f_0, f_1, \ldots, f_{s-1}$.*

Let $T_4 = \sum u_\lambda \otimes v_\lambda \otimes w_\lambda$ for $0 \leq \lambda < \mathbf{R}(T_4)$ be an optimal tensor rank decomposition. We first factor T_4 over the C space to identify single-product expressions in $A \otimes B$, and choose the independent single products $f_0 = a_{0,1}b_{1,0}$ and $f_1 = a_{0,1}b_{1,1}$. By Lemma 2, we can assume our tensor rank decomposition explicitly computes these two terms, i.e. $u_0 \otimes v_0 = a_{0,1} \otimes b_{1,0}$ and $u_1 \otimes v_1 = a_{0,1} \otimes b_{1,1}$.

Consequently, we remove these two terms $u_0 \otimes v_0 \otimes w_0$ and $u_1 \otimes v_1 \otimes w_1$ from T_4. Because we operate over $\mathbb{F}_2$, it leaves arbitrary unknown coefficients (either 0 or 1) in w_0 and w_1. Thus, the residual tensor takes the form:

$$T_4' = T_4 + a_{0,1} \otimes b_{1,0} \otimes (c_{0,0} + \mu_0 c_{0,1} + \mu_1 c_{1,1}) + a_{0,1} \otimes b_{1,1} \otimes (c_{1,0} + \mu_2 c_{0,1} + \mu_3 c_{1,1})$$

where $\mu_i \in \{0, 1\}$. By enumerating all $2^4 = 16$ possible evaluations of these coefficients, we find that $\mathbf{R}(T_4') \geq 4$ in every case by the flattening technique. Since we removed 2 rank-1 terms to reach a tensor of rank 4, we conclude $\mathbf{R}(T_4) \geq 4 + 2 = 6$.

Orbit 5: $\{a_{0,0} = 0,\ a_{1,1} = 0\}$

$A = \begin{pmatrix} 0 & a_{0,1} \\ a_{1,0} & 0 \end{pmatrix}$. The tensor is $T_5 = a_{0,1} \otimes b_{1,0} \otimes c_{0,0} + a_{0,1} \otimes b_{1,1} \otimes c_{1,0} + a_{1,0} \otimes b_{0,0} \otimes c_{0,1} + a_{1,0} \otimes b_{0,1} \otimes c_{1,1}$. Flattening a and b yields $x_{0,1,1,0} \otimes c_{0,0} + x_{0,1,1,1} \otimes c_{1,0} + x_{1,0,0,0} \otimes c_{0,1} + x_{1,0,0,1} \otimes c_{1,1}$, which has matrix rank 4. Thus, $\mathbf{R}(T_5) \geq 4$.

Orbit 6: $\{a_{0,1} + a_{1,0} = 0,\ a_{0,0} + a_{0,1} + a_{1,1} = 0\}$

$A = \begin{pmatrix} a_{0,0} & a_{0,1} \\ a_{0,1} & a_{0,0} + a_{0,1} \end{pmatrix}$. The tensor T_6 expands into 10 terms. Flattening T_6 gives a trivial matrix rank of 4, so $\mathbf{R}(T_6) \geq 4$. We can improve this using the substitution method on A.

We decompose $T_6 = \sum_{\lambda=1}^{\mathbf{R}(T_6)} u_\lambda \otimes v_\lambda \otimes w_\lambda$. The space A is generated by the basis $\{a_{0,0}, a_{0,1}\}$. Over $\mathbb{F}_2$, the only non-zero linear forms in this space are $a_{0,0}$, $a_{0,1}$, and $a_{0,0} + a_{0,1}$. Every non-zero A-component u_λ in our decomposition must be one of these three forms.

Since the decomposition has at least 4 terms, the pigeonhole principle guarantees that at least one of these three linear forms appears as u_λ at least twice (otherwise, $1+1+1 = 3 < 4$). Consequently, setting that specific linear form to 0 will eliminate at least 2 terms from the decomposition, dropping the total rank by at least 2.

- Case 1: $a_{0,0}$ appears at least twice. Setting $a_{0,0} = 0$ reduces A to $\begin{pmatrix} 0 & a_{0,1} \\ a_{0,1} & a_{0,1} \end{pmatrix}$, which is equivalent to Orbit 2 by adding the first row to the second. This yields $\mathbf{R}(T_6) \geq \mathbf{R}(T_2) + 2 \geq 4 + 2 = 6$.
- Case 2: $a_{0,1}$ appears at least twice. Setting $a_{0,1} = 0$ reduces A to $\begin{pmatrix} a_{0,0} & 0 \\ 0 & a_{0,0} \end{pmatrix}$, which is equivalent to Orbit 2 by swapping the rows. This yields $\mathbf{R}(T_6) \geq \mathbf{R}(T_2) + 2 \geq 4 + 2 = 6$.
- Case 3: $a_{0,0} + a_{0,1}$ appears at least twice. Setting $a_{0,0} + a_{0,1} = 0$ reduces A to $\begin{pmatrix} a_{0,0} & a_{0,0} \\ a_{0,0} & 0 \end{pmatrix}$, which is equivalent to Orbit 2 by adding the second row to the first. This yields $\mathbf{R}(T_6) \geq \mathbf{R}(T_2) + 2 \geq 4 + 2 = 6$.

In all cases, substitution forces the rank lower bound to 6. Hence, $\mathbf{R}(T_6) \geq 6$.

Orbit 7: $\{a_{0,0} = 0\}$

$A = \begin{pmatrix} 0 & a_{0,1} \\ a_{1,0} & a_{1,1} \end{pmatrix}$. By applying the additional restriction $a_{0,1} + a_{1,0} = 0$, we project the tensor onto the subspace corresponding to Orbit 4. Because restricting to a subspace cannot increase the tensor rank, $\mathbf{R}(T_7) \geq \mathbf{R}(T_4) \geq 6$.

Orbit 8: $\{a_{0,1} + a_{1,0} = 0\}$

$A = \begin{pmatrix} a_{0,0} & a_{0,1} \\ a_{0,1} & a_{1,1} \end{pmatrix}$. By applying the additional restriction $a_{0,0} = 0$, we project onto the subspace corresponding to Orbit 4. Thus, $\mathbf{R}(T_8) \geq \mathbf{R}(T_4) \geq 6$.

Orbit 9: $\{\}$

$A = \begin{pmatrix} a_{0,0} & a_{0,1} \\ a_{0,1} & a_{1,1} \end{pmatrix}$. This is the general unconstrained 2×2 case, $T_9 = \sum_{i=0}^{1} \sum_{j=0}^{1} \sum_{k=0}^{1} a_{i,j} \otimes b_{j,k} \otimes c_{k,i}$.

First, it is easy to show that $\mathbf{R}(T_9) \geq 1$. Then, we apply the substitution method on A. We decompose $T_9 = \sum_{\lambda=1}^{\mathbf{R}(T_9)} u_\lambda \otimes v_\lambda \otimes w_\lambda$.

If $a_{0,0}$ appears as u_λ at least once, setting $a_{0,0} = 0$ reduces A to Orbit 7. This yields $\mathbf{R}(T_9) \geq \mathbf{R}(T_7) + 1 \geq 6 + 1 = 7$. The same argument applies to $a_{0,1}$, $a_{1,0}$, $a_{1,1}$, $a_{0,0} + a_{0,1}$, $a_{1,0} + a_{1,1}$, $a_{0,0} + a_{1,0}$, $a_{0,1} + a_{1,1}$, and $a_{0,0} + a_{0,1} + a_{1,0} + a_{1,1}$ by the sandwich symmetry.

If $a_{0,1} + a_{1,0}$ appears as u_λ at least once, setting $a_{0,1} + a_{1,0} = 0$ reduces A to Orbit 8. This yields $\mathbf{R}(T_9) \geq \mathbf{R}(T_8) + 1 \geq 6 + 1 = 7$. The same argument applies to $a_{0,0} + a_{1,1}$, $a_{0,0} + a_{0,1} + a_{1,0}$, $a_{0,0} + a_{0,1} + a_{1,1}$, $a_{0,0} + a_{1,0} + a_{1,1}$, $a_{0,1} + a_{1,0} + a_{1,1}$ by the sandwich symmetry.

These are all the possible cases for u_λ. Therefore, $\mathbf{R}(\langle 2, 2, 2 \rangle) = \mathbf{R}(T_9) \geq 7$.

5 Rank Lower Bound Techniques

In this section, we formally describe the four techniques used to establish the lower bounds in our toy example.

5.1 Flattening

The tensor rank of a trilinear tensor is trivially bounded below by its matrix rank under any flattening. We flatten the tensor along its three bipartitions (grouping dimensions A and B, B and C, and C and A) and compute the respective matrix ranks. We then take the maximum of these three ranks as our flattening lower bound.

5.2 Forced Product

This technique systematically applies Hopcroft and Kerr's method (Lemma 2) to strip rank-1 terms from the decomposition. We illustrate this general algorithm using Orbit 4 as an example.

First, we group the tensor along the C factor into t terms. For T_4, this yields four terms ($t = 4$): $(a_{0,1} \otimes b_{0,0} + a_{1,1} \otimes b_{1,0}) \otimes c_{0,1}$, $(a_{0,1} \otimes b_{0,1} + a_{1,1} \otimes b_{1,1}) \otimes c_{1,1}$, $a_{0,1} \otimes b_{1,0} \otimes c_{0,0}$, and $a_{0,1} \otimes b_{1,1} \otimes c_{1,0}$. Next, we identify the terms whose $A \otimes B$ components have a matrix rank of exactly 1, meaning they can be expressed as a single product. In our example, the first two terms have rank 2, while the third $(a_{0,1} \otimes b_{1,0})$ and fourth $(a_{0,1} \otimes b_{1,1})$ have rank 1.

We greedily choose a maximal linearly independent subset of these rank-1 terms, denoting the size of this subset by s. In T_4, the two rank-1 terms are independent, so we select both ($s = 2$). By Lemma 2, any minimal bilinear algorithm computing this tensor can be assumed to explicitly compute these s single products.

We formally project these s terms out of the tensor. Since the exact C-components of these terms in the decomposition are unknown, we must introduce arbitrary coefficients for the remaining $t-s$ basis elements of C. Over $\mathbb{F}_2$, this yields $s(t-s)$ unknown binary coefficients. We enumerate all $2^{s(t-s)}$ possible coefficient assignments, compute the flattening rank lower bound for each resulting residual tensor, and take the minimum across all cases. Because this branching step is computationally expensive, we skip this Forced Product technique entirely if $s(t-s)$ is too large. Finally, we repeat this entire process for all three cyclic permutations of the tensor and take the maximum of the resulting ranks.

In practice, we skip this technique if $s(t-s) \geq 32$. To accelerate the process, the $2^{s(t-s)}$ cases are enumerated in parallel on the GPU.

5.3 Degenerate Reduction

If a set of linear restrictions X is a subset of another restriction set Y ($X \subseteq Y$), then the subspace defined by Y is contained within the subspace defined by X. Consequently, the tensor T_Y is a linear projection of T_X. Because tensor rank is monotonically non-increasing under linear projection, we have $\mathbf{R}(T_X) \geq \mathbf{R}(T_Y)$. Thus, any lower bound established for the heavily restricted Orbit Y provides a lower bound for the looser Orbit X.

5.4 Substitution with Backtracking

Substitution is a powerful inductive technique, used extensively by Pan, Hopcroft, Kerr, and Bläser. The core idea is to assume the existence of a decomposition of length r, $T = \sum_{i=1}^{r} u_i \otimes v_i \otimes w_i$, and apply a carefully chosen linear functional to one factor space (e.g., setting a specific linear combination of A-variables to zero) so that one or more rank-1 terms vanish. By analyzing the rank of the residual tensor, we can conclude an inequality of the form $\mathbf{R}(T) \geq \mathbf{R}(T') + (\text{rank drop})$.

While the substitution for Orbit 6 was constructed manually in Section 3, we present an automated backtracking algorithm to systematically discover these substitution sequences.

First, we establish a baseline lower bound, denoted *known_lower_bound*, using the previous three techniques. We enumerate all possible valid evaluations (non-zero linear forms) for the A-components u_λ in the decomposition. By setting a subset of these A-components to zero, we algebraically reduce the tensor to another, more restricted orbit.

To minimize redundant branching, we only consider canonical A-components. For example, under the restriction set $\{a_{0,1} + a_{1,0} = 0,\ a_{0,0} + a_{0,1} + a_{1,1} = 0\}$ in Orbit 6, many linear forms evaluate to the same value. Because $a_{0,1} + a_{1,0} = 0$, $a_{0,1}$ and $a_{1,0}$ are functionally equivalent. We select

the lexicographically minimal representative from each equivalence class. Specifically, a canonical A-component must not contain the variable $a_{i,j}$ if (i, j) corresponds to the leading pivot (most significant bit) of any restriction in the RREF restriction set. In Orbit 6, $(1, 0)$ is the leading pivot of the restriction $a_{0,1}+a_{1,0} = 0$, and $(1, 1)$ is the leading pivot of the restriction $a_{0,0}+a_{0,1}+a_{1,1} = 0$. Thus, canonical A-components must be formed strictly from the remaining free variables $\{a_{0,0}, a_{0,1}\}$. The valid non-zero canonical A-components are $a_{0,0}$, $a_{0,1}$, and $a_{0,0} + a_{0,1}$.

The backtracking search attempts to find a sequence of substitutions that improves the rank lower bound. For Orbit 6, with an initial flattening bound of 4, we enumerate all combinations of A-components of length 4 from our canonical set $\{a_{0,0}, a_{0,1}, a_{0,0}+a_{0,1}\}$ (assuming non-decreasing order to eliminate permutations), and try to improve the lower bound to 6. The automated backtracking below mirrors our manual proof:

- $a_{0,0}$: Lower bound of 6 not met. Go deeper.
  - $a_{0,0}$, $a_{0,0}$: Setting $a_{0,0} = 0$ reduces to Orbit 2. $\mathbf{R}(T_6) \geq \mathbf{R}(T_2) + 2 \geq 4 + 2 = 6$. Proved; backtrack.
  - $a_{0,0}$, $a_{0,1}$: Lower bound of 6 not met. Go deeper.
    - $a_{0,0}$, $a_{0,1}$, $a_{0,1}$: Setting $a_{0,1} = 0$ reduces to Orbit 2. $\mathbf{R}(T_6) \geq \mathbf{R}(T_2) + 2 \geq 4 + 2 = 6$. Proved; backtrack.
    - $a_{0,0}$, $a_{0,1}$, $a_{0,0} + a_{0,1}$: Lower bound of 6 not met. Go deeper.
      - $a_{0,0}$, $a_{0,1}$, $a_{0,0} + a_{0,1}$, $a_{0,0} + a_{0,1}$: Setting $a_{0,0} + a_{0,1} = 0$ reduces to Orbit 2. $\mathbf{R}(T_6) \geq \mathbf{R}(T_2) + 2 \geq 4 + 2 = 6$. Proved; backtrack.
  - $a_{0,0}$, $a_{0,0} + a_{0,1}$: Lower bound of 6 not met. Go deeper.
    - $a_{0,0}$, $a_{0,0} + a_{0,1}$, $a_{0,0} + a_{0,1}$: Setting $a_{0,0} + a_{0,1} = 0$ reduces to Orbit 2. $\mathbf{R}(T_6) \geq \mathbf{R}(T_2) + 2 \geq 4 + 2 = 6$. Proved; backtrack.
- $a_{0,1}$: Lower bound of 6 not met. Go deeper.
  - $a_{0,1}$, $a_{0,1}$: Setting $a_{0,1} = 0$ reduces to Orbit 2. $\mathbf{R}(T_6) \geq \mathbf{R}(T_2) + 2 \geq 4 + 2 = 6$. Proved; backtrack.
  - $a_{0,1}$, $a_{0,0} + a_{0,1}$: Lower bound of 6 not met. Go deeper.
    - $a_{0,1}$, $a_{0,0} + a_{0,1}$, $a_{0,0} + a_{0,1}$: Setting $a_{0,0} + a_{0,1} = 0$ reduces to Orbit 2. $\mathbf{R}(T_6) \geq \mathbf{R}(T_2) + 2 \geq 4 + 2 = 6$. Proved; backtrack.
- $a_{0,0} + a_{0,1}$: Lower bound of 6 not met. Go deeper.
  - $a_{0,0} + a_{0,1}$, $a_{0,0} + a_{0,1}$: Setting $a_{0,0} + a_{0,1} = 0$ reduces to Orbit 2. $\mathbf{R}(T_6) \geq \mathbf{R}(T_2) + 2 \geq 4 + 2 = 6$. Proved; backtrack.

Formally, we summarize this process in Algorithm 3. Note that it enumerates the A-components in decreasing order.

Algorithm 3 Backtracking Algorithm

Require: *restriction_set*, a set of restrictions on A, in RREF form.
Require: Best-known rank lower bound of each orbit.
Require: The target lower bound.
Ensure: Returns the rank lower bound with a *proof*.

```
1: Compute canonical_a_components
2: function BACKTRACK(a_components)
3:     if a_components is not empty then
4:         for each sub-list sub_a_components of a_components containing the last component do
5:             extended_restrictions ← restriction_set ∪ sub_a_components
6:             Find the orbit of extended_restrictions by enumerating symmetries and applying
   Gauss-Jordan elimination
7:             if |sub_a_components| + best-known rank lower bound of extended_restrictions ≥
   target lower bound then
8:                 Save |sub_a_components| and the symmetry to proof
9:                 return True
10:            end if
11:        end for
12:    end if
13:    last_component ← the last element of a_components, or 2^{lm} − 1 if a_components is empty
14:    for component in canonical_a_components such that component ≤ last_component do
15:        if BACKTRACK(a_components + [component]) is False then
16:            return False
17:        end if
18:    end for
19:    return True
20: end function
```

5.4.1 Optimizations

Algorithm 3 is sufficient for small formats like $\langle 2, 2, 2\rangle$, but requires significant optimization to scale to larger matrices.

Multi-threading: Because the backtracking for an orbit depends strictly on the lower bounds of orbits with more restrictions, we process them from the most restricted orbits to the general unconstrained case. For each number of restrictions, we process the orbits with that number of restrictions in parallel. Also, the a-component enumeration loop at the first depth of the backtracking tree is parallelized.

Symmetry Orbit Hash Map: In Algorithm 3, identifying the reduced orbit (Line 6) requires enumerating all full-rank matrices P, Q^{-1}, and transpositions, which is a major computational bottleneck. We optimize this by precomputing a hash map for each orbit that stores its right-multiplied and transposed configurations, just like the optimization idea in Algorithm 2. During

the search, we only need to enumerate the left-multiplication matrices P, yielding a roughly square-root speedup in lookup time. To avoid lock contention during parallel inserts, the hash map is sharded (e.g., into 997 shards) using fine-grained mutex locks.

Thread-Local Cache: Even with the optimized map, enumerating P remains expensive. We introduce a thread-local cache for each orbit that maps *sub_a_components* directly to their previously computed lower bounds. To limit the memory usage, when a local cache exceeds a threshold (typically 3×10^6 elements), we randomly evict half of its entries.

Incremental Target Strategy: Rather than immediately searching for the ultimate target bound (like 4 to 6 in the example), we iteratively set the target lower bound to the best-known lower bound plus one (e.g. 4 to 5, then 5 to 6). This incremental approach makes each step easier and increases the chance of pruning.

Step Limits: To prevent very slow search on difficult branches, we enforce a strict step limit. If the backtracking algorithm exceeds this limit, the entire backtracking search is aborted.

6 Proof Certificates and Verification

To ensure our computational results are quickly verifiable, our framework generates a formal proof certificate for every established lower bound. For each restriction orbit, the certificate records the specific technique successfully used to establish its lower bound.

Depending on the technique, we record the following minimal information required for a verifier program to reconstruct the proof quickly:

- **Flattening:** Nothing is recorded. It runs fast.
- **Forced Product:** We record the cyclic permutation index (in $\{0, 1, 2\}$) under which the rank-1 terms were identified and projected out.
- **Degenerate Reduction:** We record the additional restrictions (the set difference $Y \setminus X$) that project the current orbit X into the strictly smaller subspace Y.
- **Substitution with Backtracking:** Because full backtracking trees are prohibitively large, we optimize the trade-off between certificate size and verification time. Specifically, at the point a target bound is successfully reached (Line 9 in Algorithm 3), we record:
  1. the size of the sequence *a_components*;
  2. the specific index set of the *sub_a_components* within *a_components*;
  3. the exact symmetry transformation (the base change matrices P, Q^{-1}, and the transpose Boolean flag) mapping the substituted residual tensor to a known canonical orbit.

Since the lower bound proof for an orbit only depends on orbits with more restrictions, we verify the certificates from the most restricted orbits to the general one. We accelerate this procedure by parallelizing the verification across all orbits with the same number of restrictions.

7 Applications

Using the automated framework, we established several tensor rank lower bounds over $\mathbb{F}_2$.

7.1 $\mathbf{R}(\langle 3,3,3\rangle) \geq 20$ over $\mathbb{F}_2$

Using a step limit of 10^7, our search algorithm found the proof in 71 minutes on an Apple MacBook Air M4 with 8 cores and 16 GB of memory. The generated proof certificate is approximately 32 MiB in size and can be verified on the same hardware in less than 10 seconds.

7.2 $\mathbf{R}(\langle 2,3,4\rangle) \geq 19$ over $\mathbb{F}_2$

By cyclic symmetry, $\mathbf{R}(\langle 2,3,4\rangle) = \mathbf{R}(\langle 3,2,4\rangle)$. Our program directly evaluates the $\langle 3,2,4\rangle$ format. With a step limit of 10^7, the search phase took 110 minutes on the MacBook Air.

The resulting proof certificate is under 100 KiB. Because the proof uses the Forced Product technique at large scale, verification takes roughly 110 minutes on the same machine only using the CPU. Although the verification time matches the search time, it still remains valuable: the verifier program is significantly shorter and simpler than the search algorithm, making it much easier to independently audit for logical correctness. The verification can be accelerated by an NVIDIA L20 GPU to approximately 1 minute.

7.3 $\mathbf{R}(\langle 3,3,4\rangle) \geq 25$ over $\mathbb{F}_2$

For this larger format, we increased the step limit to 10^8. The search was run on a cloud server equipped with 64 CPU cores and 128 GB of memory, taking 3.5 days to complete (at a compute cost of roughly 168 USD[2]).

The proof certificate is approximately 626 MiB. Verification takes roughly 3.5 hours on the same CPU server (7 USD[2]), or just 32 minutes using an NVIDIA L20 GPU (1.3 USD[3]).

[2] Alibaba Cloud. Elastic Compute Service. ecs.c9ae.16xlarge. AMD EPYC 9T95.
[3] Alibaba Cloud. Elastic Compute Service. ecs.gn8is.4xlarge.

7.4 $\mathbf{R}(\langle 3,4,4\rangle) \geq 29$ over $\mathbb{F}_2$

For the $\langle 3,4,4\rangle$ format, we used a step limit of 8×10^5. The search completed in roughly 7 hours on a larger cloud server featuring 192 CPU cores and 384 GB of memory (costing about 42 USD[4]).

This produced a 2 GiB proof certificate. It can be verified on the same 192-core machine in about 3.3 hours (20 USD[4]), or on an NVIDIA L20 GPU in approximately 2.5 hours (5.4 USD[3]).

7.5 Code and Data

The complete source code for the search and verification algorithms, along with the generated proof certificates, are publicly available at:
`https://github.com/wcgbg/matrix-multiplication-lower-bound-n3r20f2`.

8 Conclusion and Open Problems

In this paper, we presented an automated framework for establishing lower bounds on the bilinear complexity of small matrix multiplication over $\mathbb{F}_2$. By systematically combining restriction enumeration, symmetry reduction, and a highly optimized backtracking search for substitution sequences, we obtained several new bounds. Most notably, our computer search program successfully proved that $\mathbf{R}(\langle 3,3,3\rangle) \geq 20$ over $\mathbb{F}_2$, improving upon a longstanding lower bound. This demonstrates that computational methods can effectively navigate the combinatorial explosion inherent in tensor rank decompositions when the search space is rigorously pruned using algebraic symmetries and canonical forms. Furthermore, this framework also applies to the multiplication of structured matrices, such as triangular, symmetric, and skew-symmetric matrices.

Several natural directions for future research remain open. While the results presented here focus on $\mathbb{F}_2$, the underlying methodology can be adapted for other finite fields. As a proof of concept, our framework successfully proved that $\mathbf{R}(\langle 2,2,2\rangle) \geq 7$ over $\mathbb{F}_3$; however, discovering new lower bounds over fields of different characteristics is a problem we leave for future work.

Second, although we successfully evaluated formats up to $\langle 3,4,4\rangle$, scaling the search to larger tensors like $\langle 4,4,4\rangle$ remains out of reach due to the huge number of orbits. Discovering stronger theoretical constraints, or more aggressive algorithmic symmetry-breaking techniques to further compress the search space, is a critical next step for scaling these methods.

Finally, the exact tensor rank of $\langle 3,3,3\rangle$ over $\mathbb{F}_2$ is still an open question, currently constrained between our new lower bound of 20 and the known upper bound of 23. Closing this gap completely remains a compelling challenge for future computational algebraic complexity research.

[4] Alibaba Cloud. Elastic Compute Service. ecs.c9ae.48xlarge. AMD EPYC 9T95.

References

[1] Markus Bläser. A $5/2n^2$-lower bound for the multiplicative complexity of $n \times n$-matrix multiplication. In *Proceedings of the 40th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 45–50. IEEE, 1999.

[2] Markus Bläser. Lower bounds for the multiplicative complexity of matrix multiplication. *computational complexity*, 8(3):203–226, 1999.

[3] Markus Bläser. On the complexity of the multiplication of matrices of small formats. *Journal of Complexity*, 19(1):43–60, 2003.

[4] Nader H Bshouty. A lower bound for matrix multiplication. *SIAM Journal on Computing*, 18(4):759–765, 1989.

[5] Luca Chiantini, Christian Ikenmeyer, Joseph M Landsberg, and Giorgio Ottaviani. The geometry of rank decompositions of matrix multiplication i: 2×2 matrices. *Experimental Mathematics*, 28(3):322–327, 2019.

[6] Hans F de Groote. On varieties of optimal algorithms for the computation of bilinear mappings i. the isotropy group of a bilinear mapping. *Theoretical Computer Science*, 7(1):1–24, 1978.

[7] John E Hopcroft and Leslie R Kerr. On minimizing the number of multiplications necessary for matrix multiplication. *SIAM Journal on Applied Mathematics*, 20(1):30–36, 1971.

[8] Kirill Khoruzhii, Patrick Gelß, and Sebastian Pokutta. Faster algorithms for structured matrix multiplication via flip graph search, 2025.

[9] Julian David Laderman. A noncommutative algorithm for multiplying 3×3 matrices using 23 multiplications. *Bulletin of the American Mathematical Society*, 82(1):126–128, January 1976.

[10] Laker Newhouse, Dakota Goldberg, and Ricardo Ruiz. Faster symmetric matrix multiplication with thunderkittens, 2024.

[11] Alexey V Smirnov. The bilinear complexity and practical algorithms for matrix multiplication. *Computational Mathematics and Mathematical Physics*, 53(12):1781–1795, 2013.

[12] Edgar Solomonik and James W Demmel. Contracting symmetric tensors using fewer multiplications. 2015.

[13] Volker Strassen. Gaussian elimination is not optimal. *Numerische Mathematik*, 13(4):354–356, August 1969.

[14] Shmuel Winograd. On multiplication of 2×2 matrices. *Linear algebra and its applications*, 1971.